

Recurrent Neural Networks (RNN)

Rowel Atienza
rowel@eee.upd.edu.ph
University of the Philippines
Updated: 2025

Recurrent Neural Network (RNN)

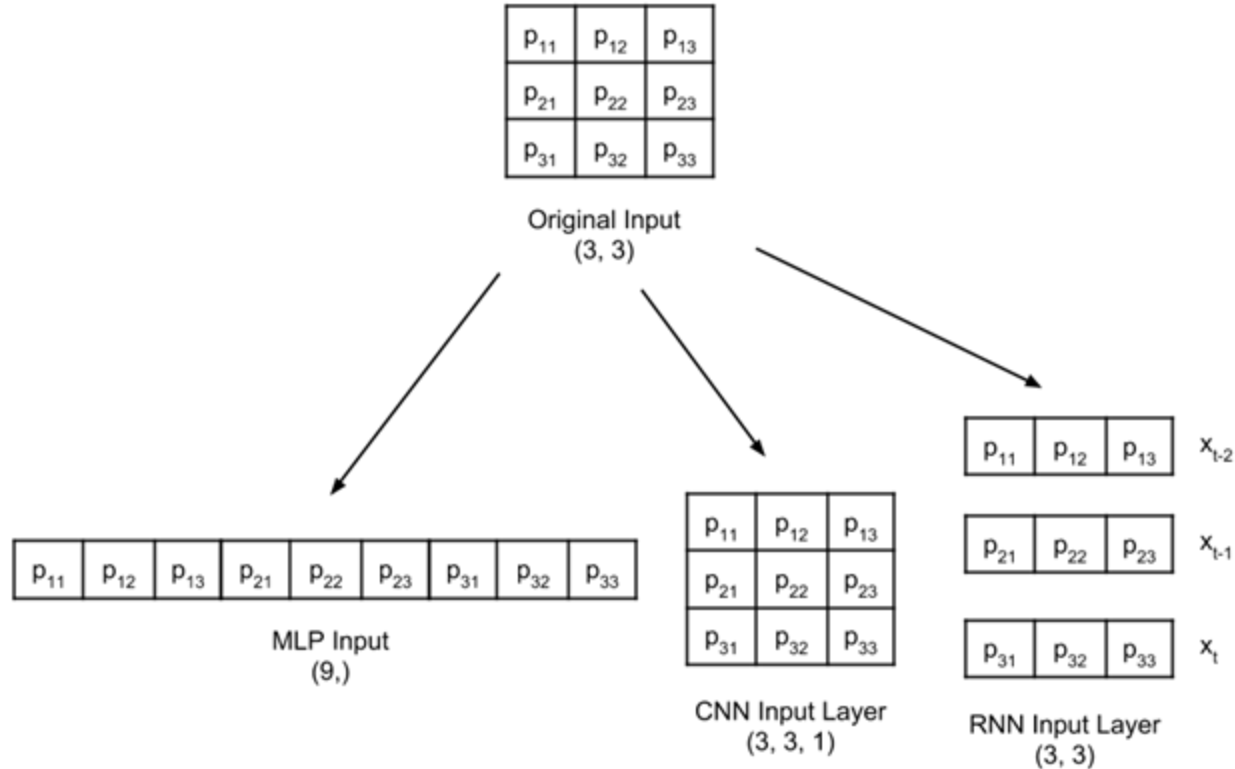
Specializes in processing **sequential data**: $x_0, x_1, \dots x_t$

Sequence of words : the quick brown fox jumps over the lazy dog (NLP)

Sequence of signals : series of quantized audio signal for speech to text (ASR)

Sequence of pixels : 2D image can be converted into 1D array of pixels

An image can be interpreted in 3 different ways

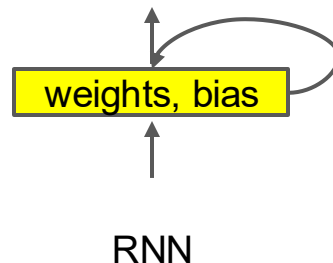
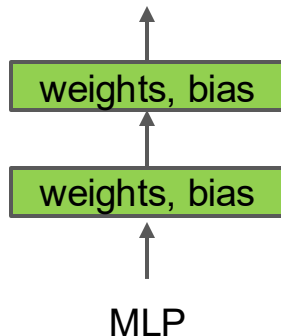
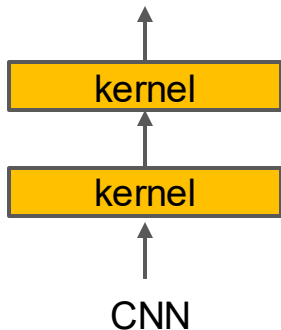


Recurrent Neural Network

One difference between RNN and CNN and MLP is parameters to be learned are shared among inputs

$x_0, x_1, \dots, x_t$ share the same set of parameters over the sequence

CNN and MLP share the same set of parameter over the entire input



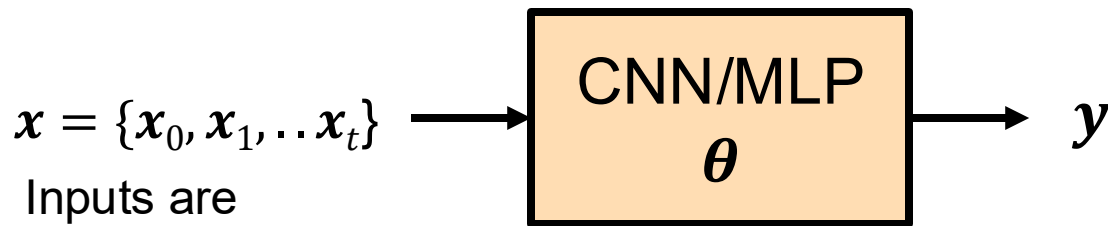
Recurrent Neural Network

Given a sequential data: $\mathbf{x} = \{x_0, x_1, \dots x_t\}$

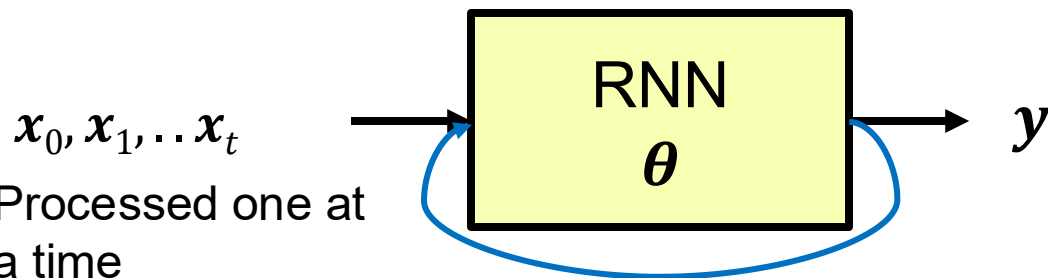
In MLP and CNN, $\mathbf{x} = \{x_0, x_1, \dots x_t\}$ are processed altogether as one in that order. 1 step.

In RNN, $\mathbf{x} = \{x_0, x_1, \dots x_t\}$ are processed one after another in a specific order. t steps.

For example:
 x is an image
made of rows
of pixels x_i



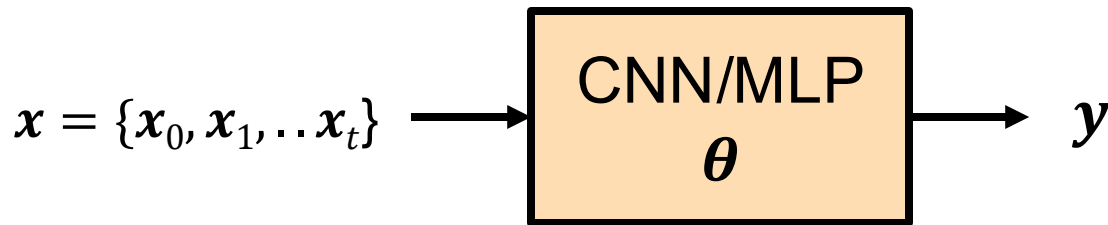
Inputs are
processed as one
Same order all the
time



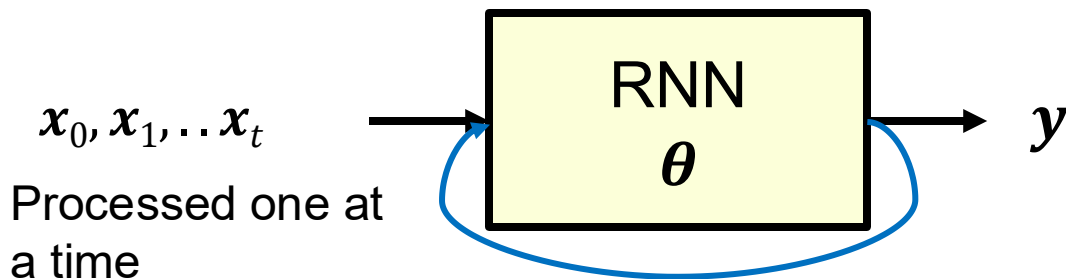
Processed one at
a time
Same order all
the time

Feedback

$$y = f(x) \approx f_n(\cdot; \theta_n) \circ f_{n-1}(\cdot; \theta_{n-1}) \circ \dots \circ f_1(x; \theta_1)$$



$$y_i = f(s_i, x_i; \theta) \quad \text{for } i = 1, \dots, t$$



Recurrent Neural Network - Unfolding

Consider a dynamical system with states: $s_0, s_1, \dots, s_t$

$$s_t = f(s_{t-1}; \theta)$$

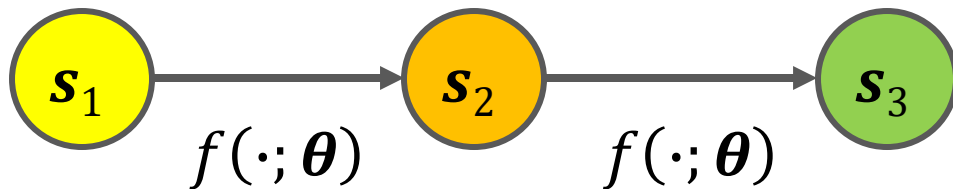
θ are the parameters

For example:

$$s_3 = f(s_2; \theta) = f(f(s_1; \theta); \theta)$$

This process is called **unfolding**

Recurrent Neural Network - Unfolding in Graphical Model



Recurrent Neural Network - with external signal

The dynamical system can also have external input: x_t

$$s_t = f(s_{t-1}, x_t; \theta)$$

If we use a neural network to store the state, a **hidden layer** h_t can be modeled as:

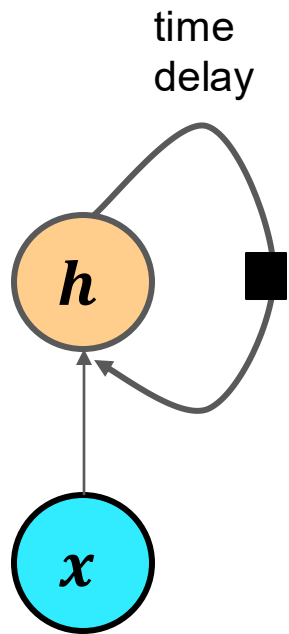
$$h_t = f(h_{t-1}, x_t; \theta)$$

The output at time t is a function of previous output of hidden layer and the current input

h_t encodes (lossy) all previous inputs $x_0, x_1, \dots, x_t$

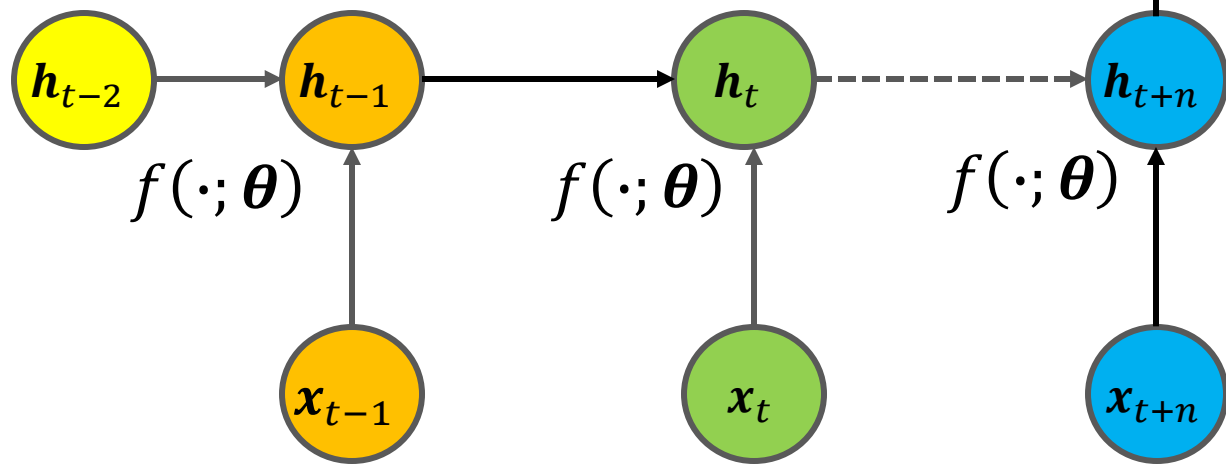
RNN Design Patterns

Unfolding of RNN (Graphical Model)

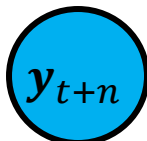


Sequence to Vector RNN

Application: classification

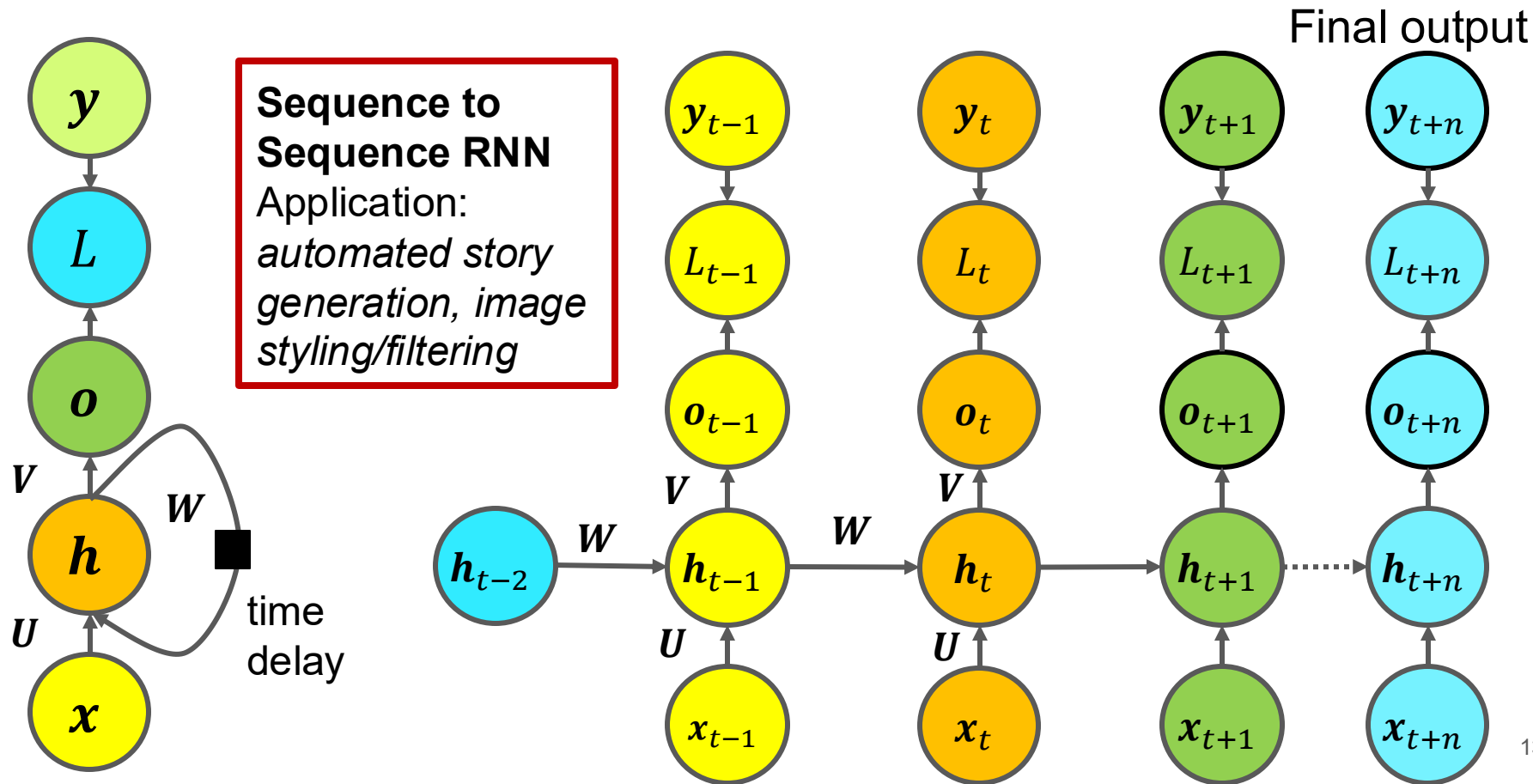


Final output



External signal

seq2seq RNN - Recurrence between hidden layers



seq2seq RNN - Recurrence between hidden layers

$$\mathbf{a}_t = \mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t)$$

$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

$$L_t = -\log p(\mathbf{y}_t | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\})$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t)$$

$\theta = \mathbf{W}, \mathbf{U}$ and $\mathbf{V}$ are parameter matrices + $\mathbf{b}$ and $\mathbf{c}$ are biases

In Pytorch, RNN implementation is slightly different. See:

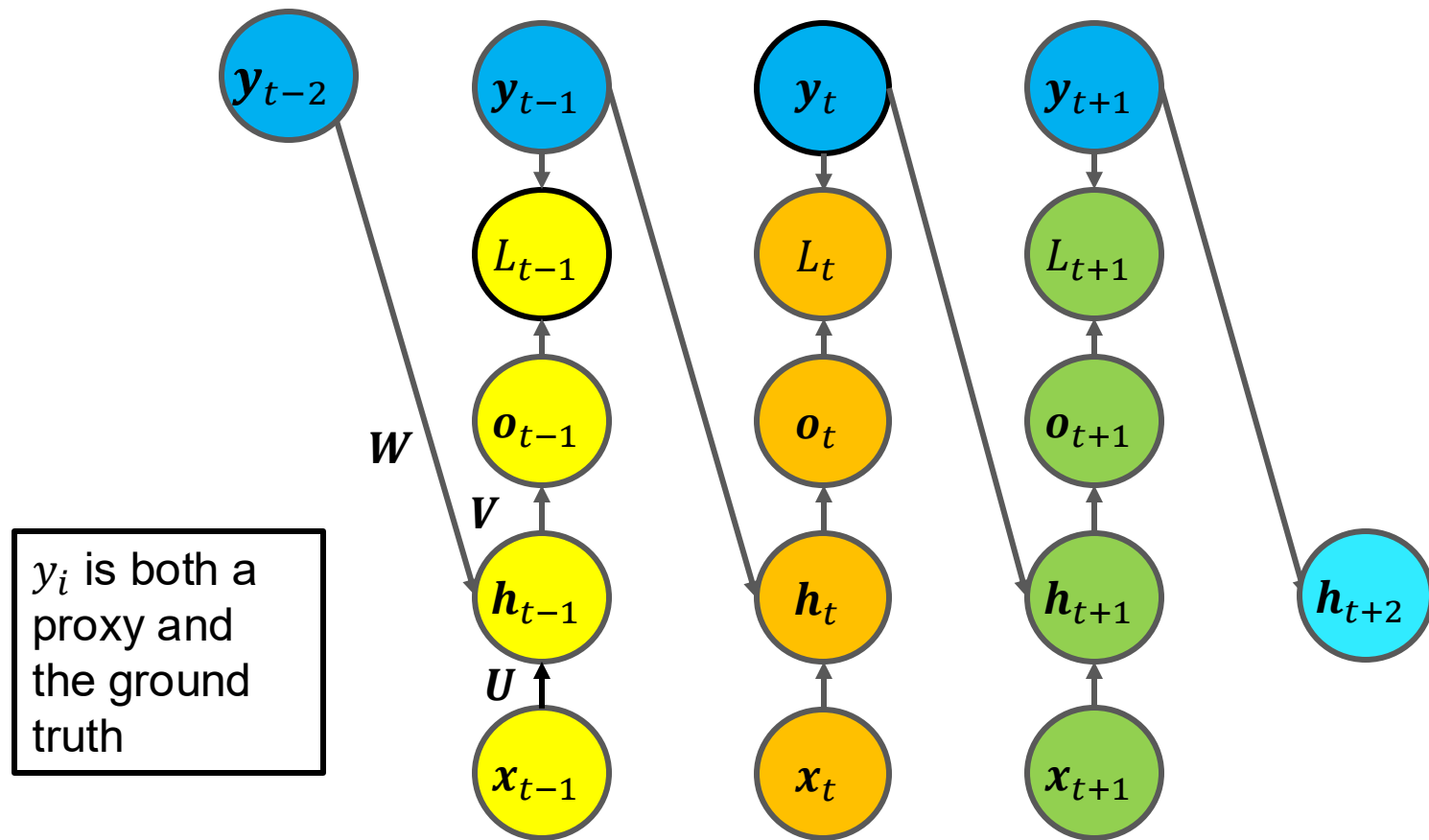
<https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>

RNN maximizes the conditional probability to minimize L

$$L_t = -\log p(\mathbf{y}_t | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\})$$

By nature, RNN is not parallelizable

seq2seq RNN - Teacher forcing (Parallelizable RNN)



Teacher forcing maximizes the conditional probability to minimize L

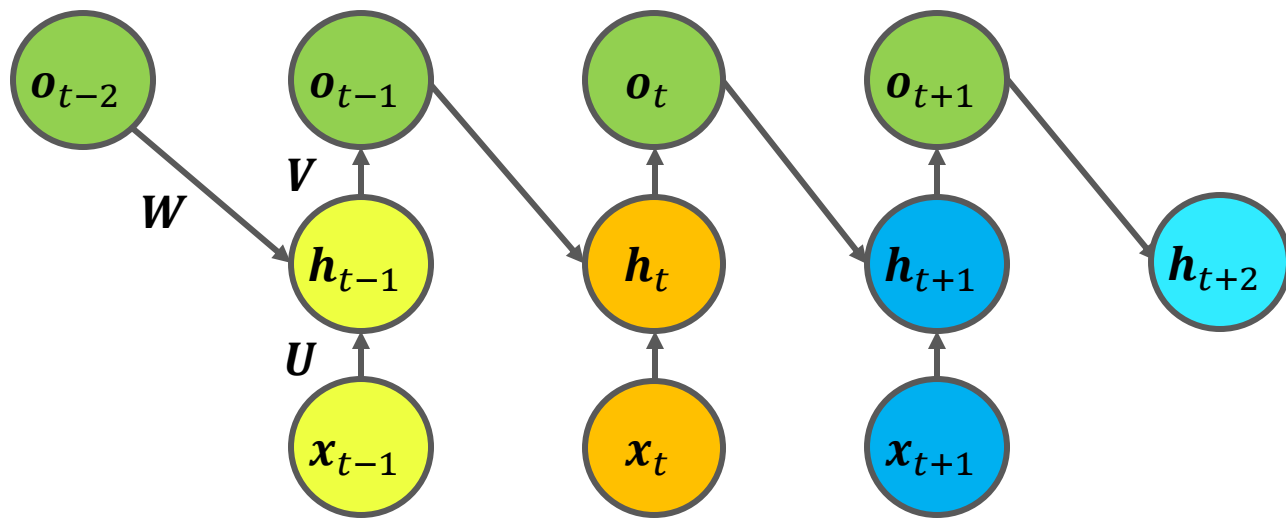
$$L_t = -\log p(\mathbf{y}_t, \mathbf{y}_{t-1} | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\})$$

By Bayes Theorem

$$L_t = -\log p(\mathbf{y}_t | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t, \mathbf{y}_{t-1}\}) - \log p(\mathbf{y}_{t-1} | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\})$$

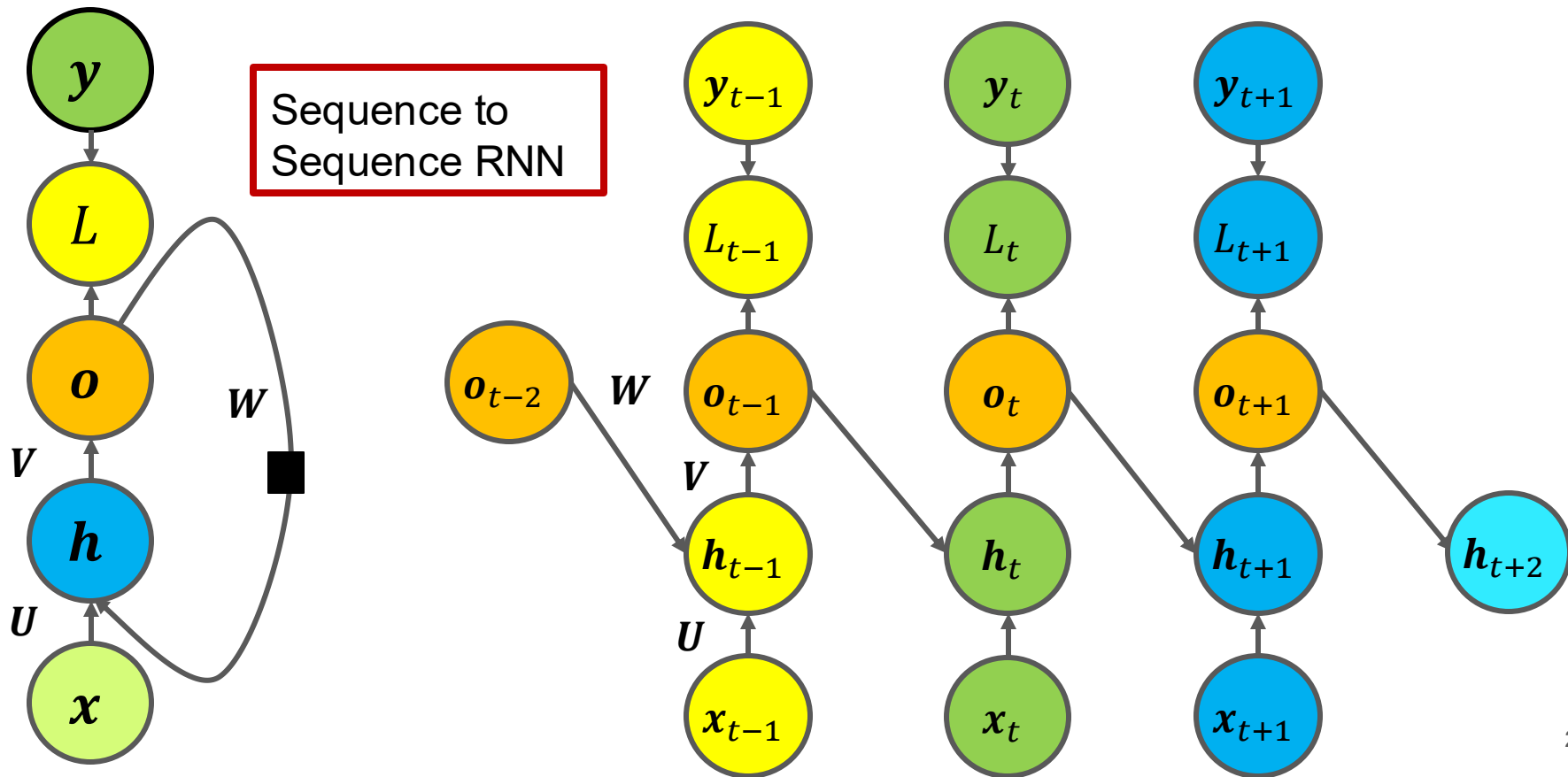
seq2seq RNN - Teacher forcing (testing)

Is the inference parallel?



Since we have no access to training outputs, we use our predicted outputs

Another variation of teacher forcing - Recurrence between output & hidden layers. Can be parallelized! o_t can be estimated by y_t during training



Teaching forcing RNN - Recurrence between output & hidden layers

Recall:

$$\mathbf{a}_t = \mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t)$$

$$\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$$

$$L = -\log p(\mathbf{y}_t | \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\})$$

$$\hat{\mathbf{y}}_t = \text{softmax}(\mathbf{o}_t)$$

$\mathbf{W}$, $\mathbf{U}$ and $\mathbf{V}$ are parameter matrices

$\mathbf{b}$ and $\mathbf{c}$ are biases

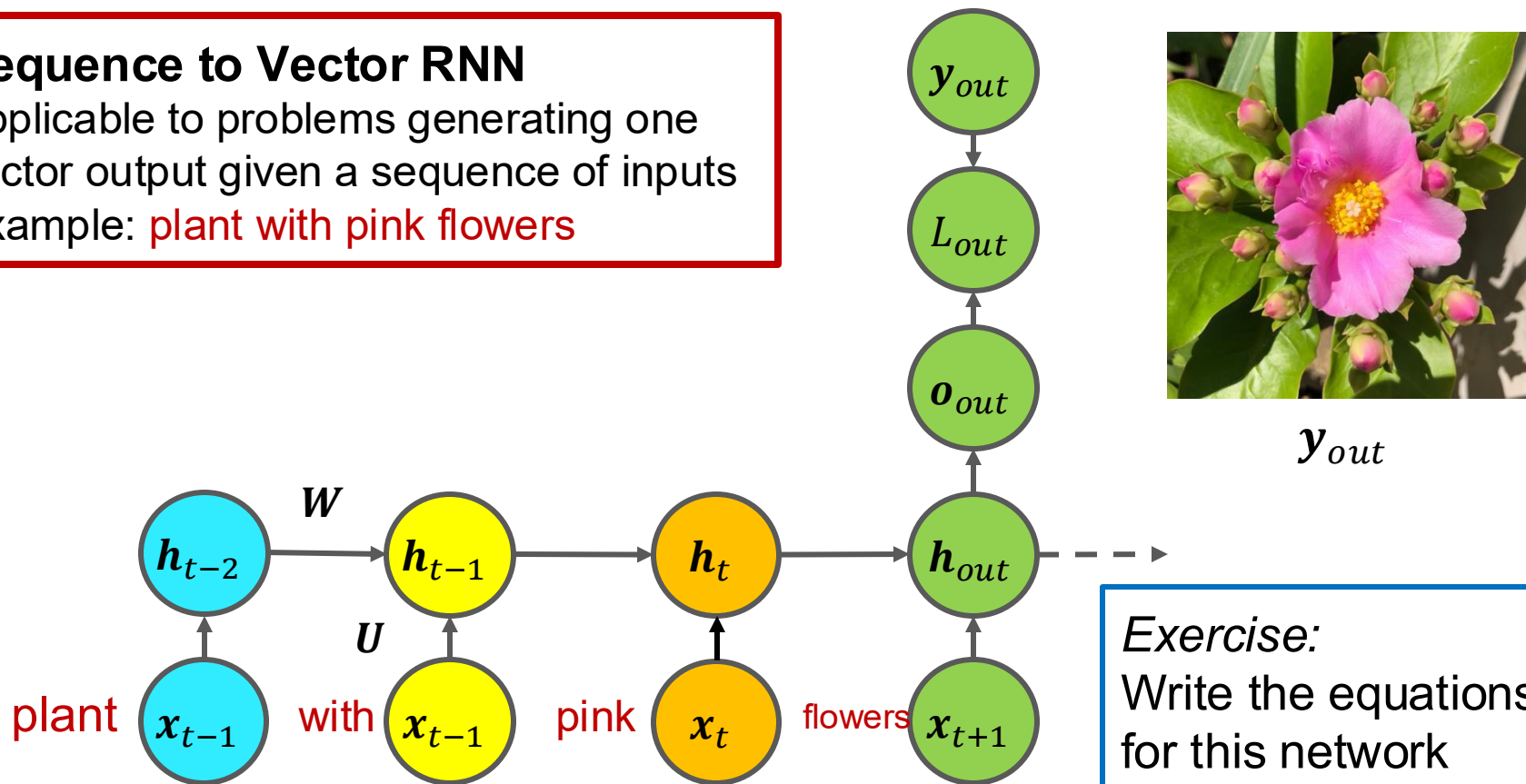
Exercise: Modify the equation for the preceding figure

seq2vec RNN

Sequence to Vector RNN

Applicable to problems generating one vector output given a sequence of inputs

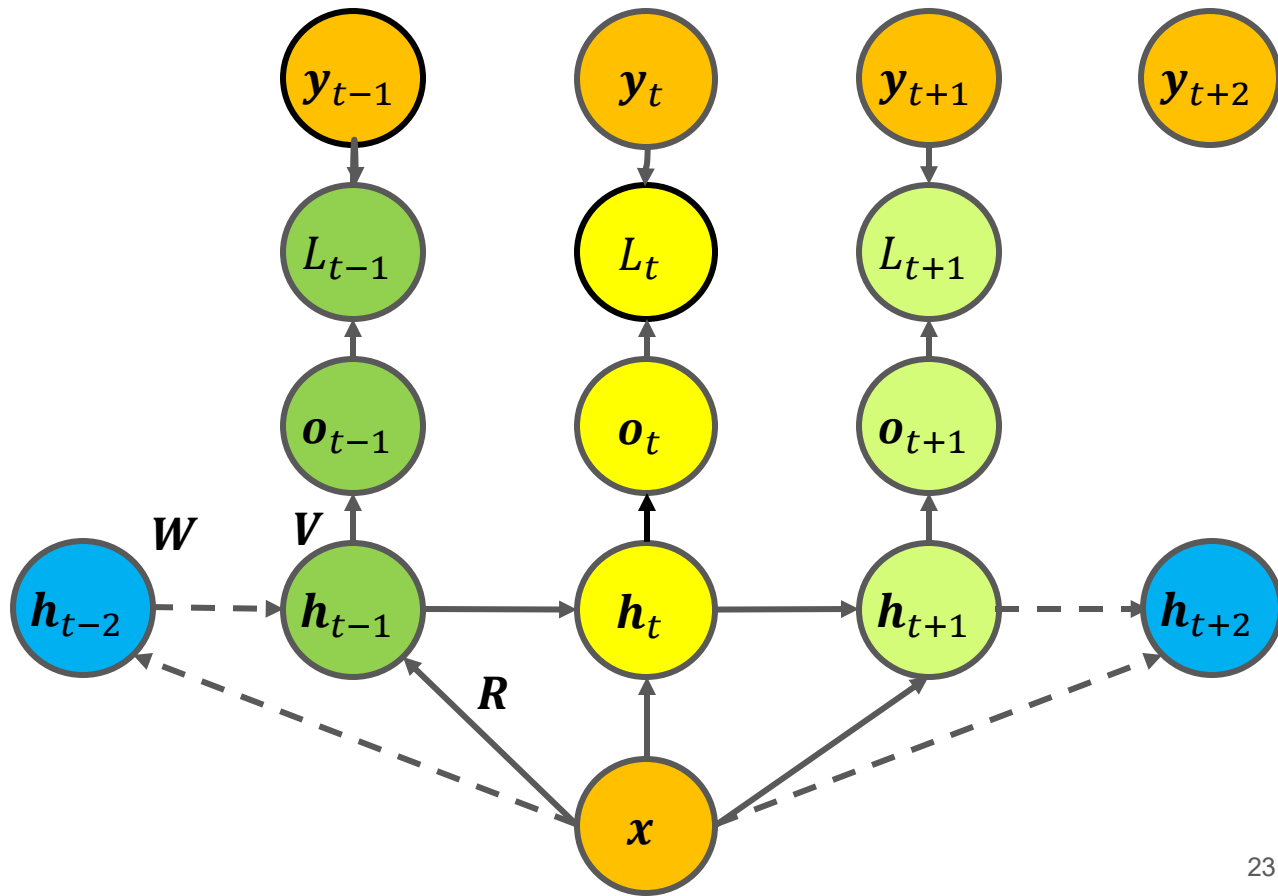
Example: **plant with pink flowers**



vec2seq RNN

y_0 y_1 y_2 y_1
A ripe calamansi fruit

Caption generation

 x 

vec2seq RNN

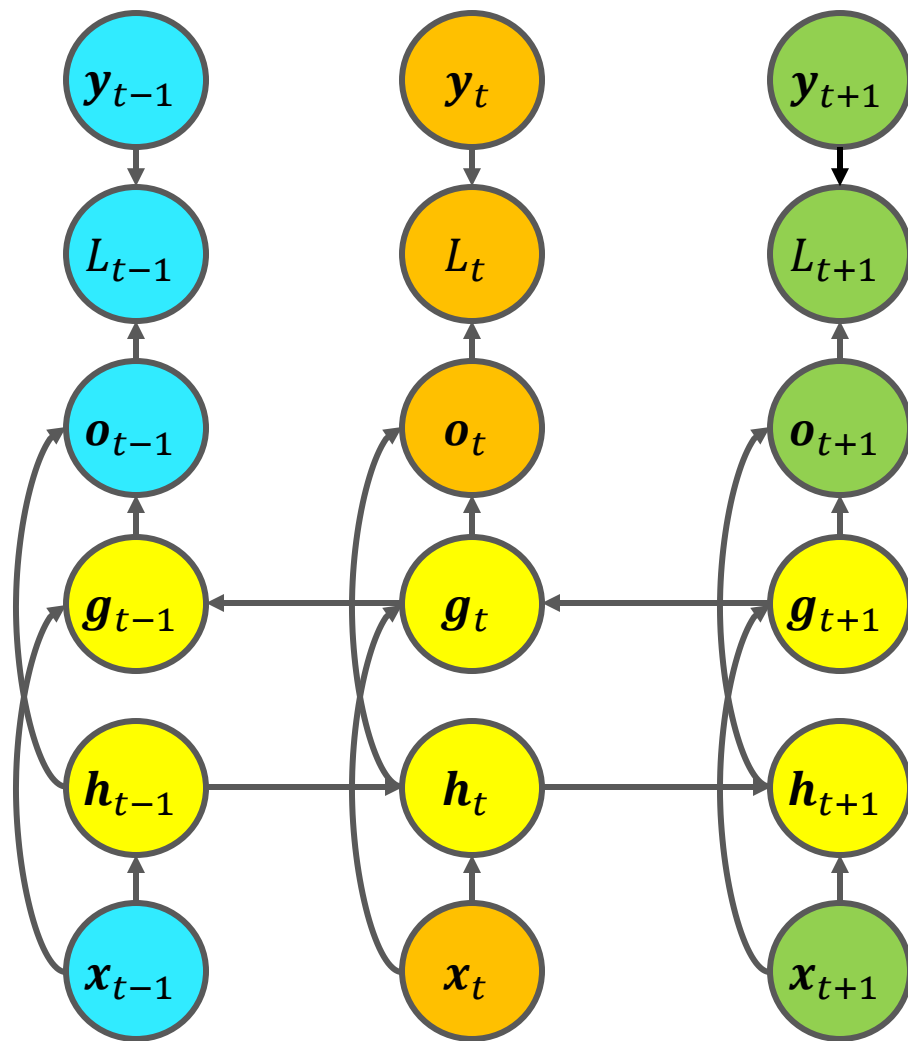
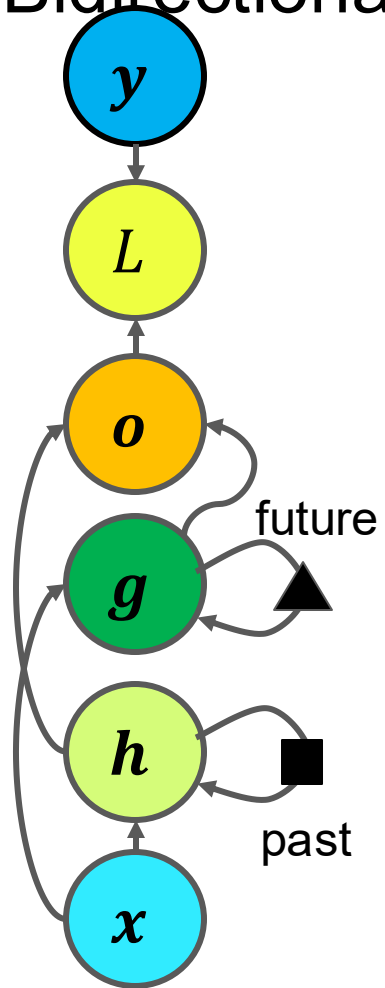
Applicable to problems converting fixed data to a variable length sequence

Image (fixed) → Caption (variable)

See <http://visualqa.org>

for good problems to solve.

Bidirectional RNN



Bidirectional RNN

Prediction is not only dependent on the past but also on the future

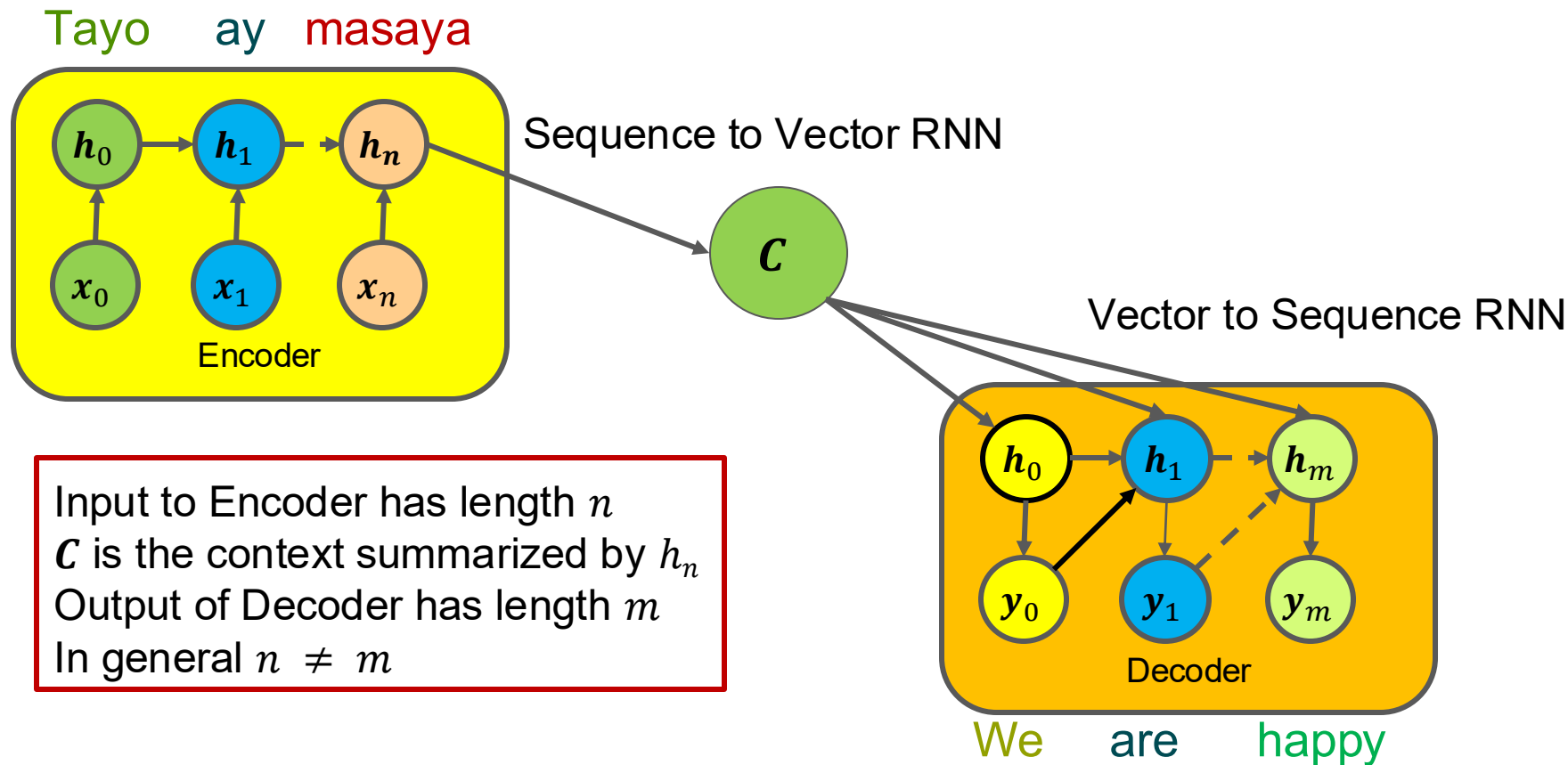
RNN prediction: *bring a golden* → ring

Even if RNN sees the next word is chicken, there is no way to correct ring

Bi RNN prediction: *bring a golden* → spring ← chicken

Applicable to problems where correction/disambiguation on the prediction is needed: Speech to text, Video understanding, NLU

Encoder-Decoder Sequence-to-Sequence



Encoder-Decoder Sequence-to-Sequence

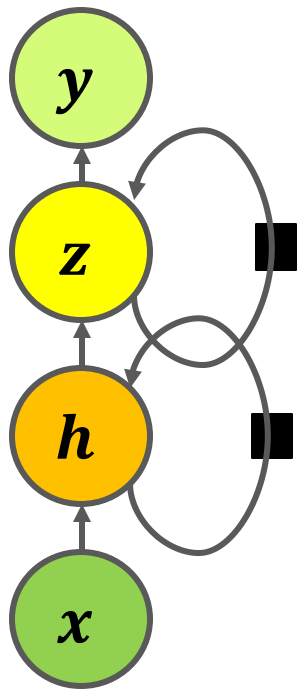
A vector, x , is encoded into context, C . The information in C is used to decode output vector, y .

Applicable in problems generating a sequence from a given sequence:

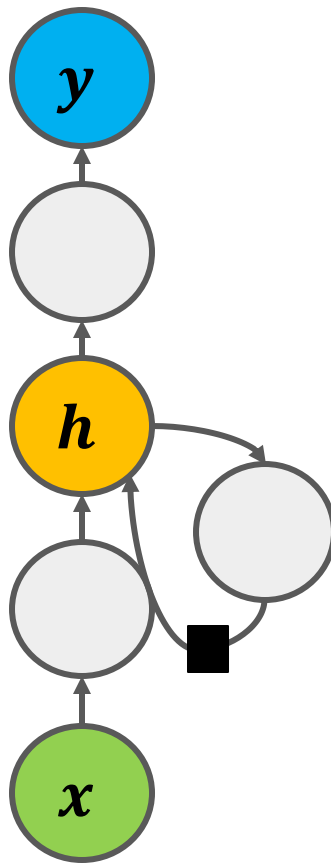
Language translation, speech to text, question-answering, etc

Deep RNN

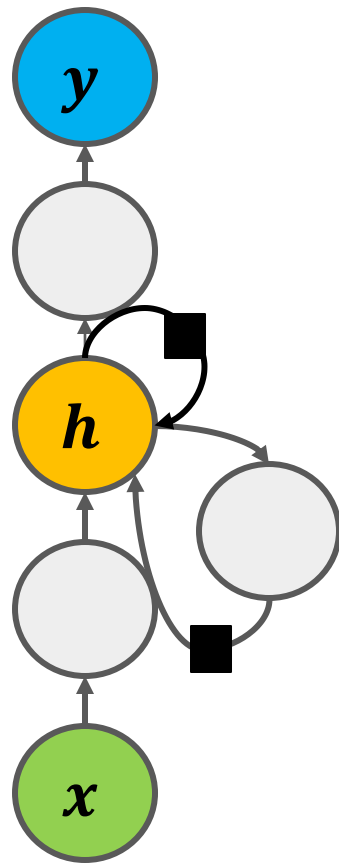
Deep Hidden Layer



MLP in input, hidden
and output layers



Skip connection



Problem with Long-Term Dependency

Consider: $\mathbf{h}_t = \mathbf{W}\mathbf{h}_{t-1} = \mathbf{W}^t\mathbf{h}_0 = \mathbf{Q}^T\mathbf{\Lambda}^t\mathbf{Q}\mathbf{h}_0$

As $t \rightarrow \infty$, eigenvalues < 1.0 will decay to zero while those > 1.0 will explode in time

Problem with Long-Term Dependency

How to deal with long-term dependency problems

Time-delay e.g. $t \rightarrow t/2$

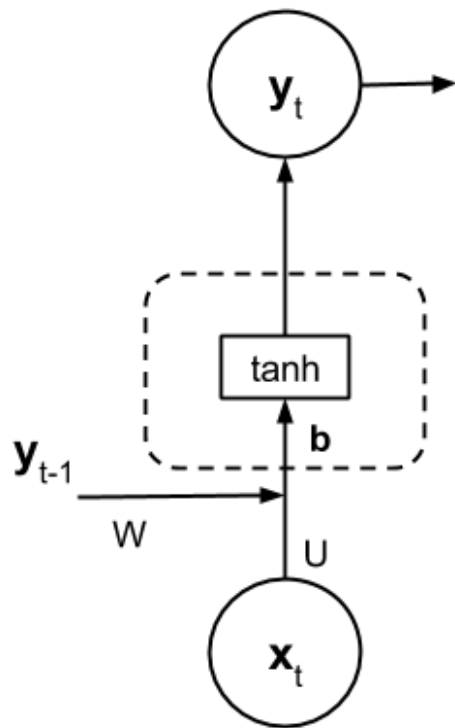
Skip connections/Remove connections

Leaky Unit: $u_t = \alpha u_{t-1} + (1 - \alpha)v_t$ where $\alpha \in [0.0, 1.0]$, u is running average

Gradient clipping

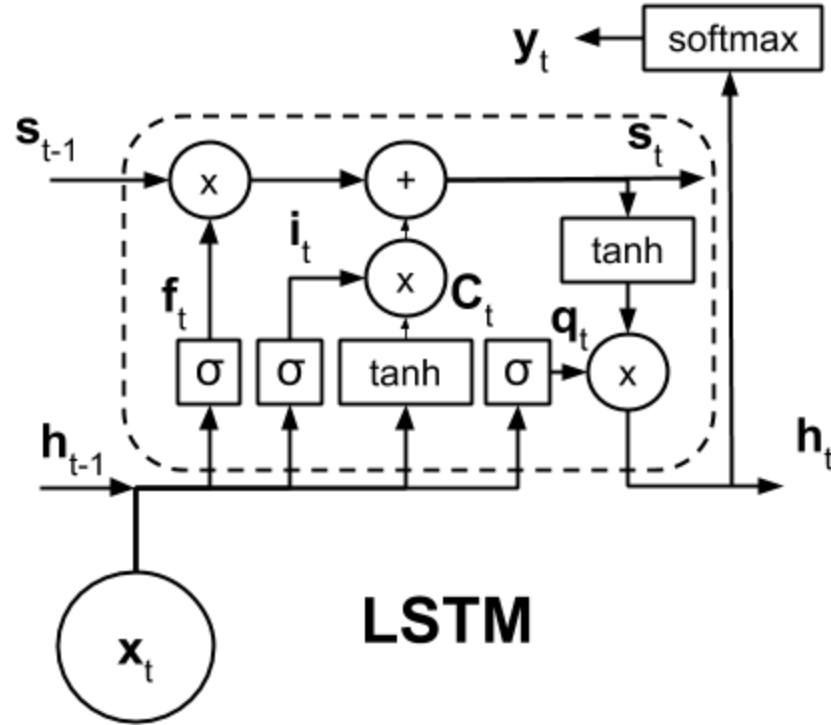
Use LSTM or GRU

RNN Cell w/ Teacher Forcing



LSTM - Long Short-Term Memory

f_t : forget gate
 i_t : input gate
 q_t : output gate



Reference

Deep Learning, Ian Goodfellow and Yoshua Bengio and Aaron Courville, MIT Press, 2016, <http://www.deeplearningbook.org>

Kapathy, A. The Unreasonable Effectiveness of RNN, <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Olah, C. Understanding LSTM Networks, <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

In Summary

RNN family excels in learning patterns in sequential data
RNN layers are inherently slow and tricks are needed to parallelize

Transformer network has emerged as a superior replacement of RNN

seq2seq

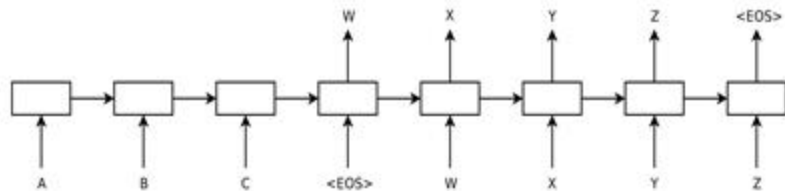


Figure 1: Our model reads an input sentence “ABC” and produces “WXYZ” as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

[2015 Sutskever et al Sequence to Sequence Learning with Neural Networks]